

Simulação de um Sistema Operacional Multitarefa de Tempo Compartilhado

Projeto A - Sistemas Operacionais

Nome: Cecília Fischer Moreira

RA:2659298

Nome: Lucas Contieri Schaefer

RA:2660989

Link do GitHub do projeto: <https://github.com/fmceci/Sistemas-Operacionais---Projeto-A>

Objetivo do projeto:

O simulador desenvolvido tem como objetivo representar visualmente o comportamento de um sistema operacional multitarefa preemptivo, permitindo acompanhar a execução de tarefas ao longo do tempo.

Tela 1 — Inicialização do Sistema

```
=====
Simulador de SO Multitarefa - v0.5
=====
```

Elemento visual

Origem

Nome do simulador

Definido no main.c

Tela 2 — Configurações da 'entrada.txt'

```
Configuracoes carregadas de 'entrada.txt':  
  Algoritmo : PRIOP  
  Quantum   : 5  
  CPUs      : 2
```

Essa tela apresenta as configurações iniciais do simulador carregadas a partir do arquivo entrada.txt. Essas informações são lidas pela função load_config, localizada no arquivo config.c, e posteriormente exibidas na main.c. A função load_config é responsável por abrir, interpretar e armazenar os dados do arquivo de configuração nas estruturas internas do sistema.

Elemento visual	Origem
Configuracoes carregadas de 'entrada.txt'	Nome do arquivo informado na função load_config
Algoritmo : PRIOP	Campo algorithm da struct Config, preenchido pela leitura da primeira linha do arquivo
Quantum : 5	Campo quantum da struct Config, carregado do arquivo entrada.txt
CPUs : 2	Campo cpu_count da struct Config, definido na leitura do arquivo
Valores exibidos na tela	Função load_config localizada em config.c
Estrutura que armazena as configurações	Struct Config
Estrutura que armazena as tarefas	Vetor de structs Task
Leitura da Linha 1 do arquivo	algoritmo_escalonamento;quantum;qtde_cpus
Leitura das linhas seguintes	id;cor;ingresso;duracao;prioridade;lista_de_eventos

Exibição das
configurações no
terminal

Função main em main.c

Tela 3 — Seleção de Configurações

```
Selecione o algoritmo (ENTER para manter PRIOP):  
  1 - SRTF (Shortest Remaining Time First)  
  2 - PRIOP (Prioridade Preemptivo)  
Opcao: 1  
Quantum (ENTER para manter 5):  
Numero de CPUs (ENTER para manter 2):
```

Essa tela permite que o usuário altere algumas configurações carregadas inicialmente do arquivo entrada.txt antes do início da simulação. Mesmo que o arquivo já possua valores definidos para algoritmo, quantum e quantidade de CPUs, o programa oferece a possibilidade de manter esses valores pressionando ENTER ou substituí-los por novos valores digitados no terminal.

O primeiro parâmetro configurável é o algoritmo de escalonamento. O sistema exibe as opções disponíveis: SRTF, que representa o algoritmo Shortest Remaining Time First, e PRIOP, que representa o escalonamento preemptivo por prioridade. Caso o usuário digite uma opção válida, o campo algorithm da estrutura Config é atualizado. Caso contrário, o programa mantém o algoritmo previamente carregado.

Em seguida, o simulador permite alterar o quantum, que representa o tempo máximo que uma tarefa pode executar antes de uma possível troca de contexto. Esse valor é lido pela função ler_inteiro_com_padrao, que permite manter o valor anterior ao pressionar ENTER. Caso o usuário informe um valor menor que 1, o programa corrige automaticamente para 1.

Por fim, a tela permite configurar a quantidade de CPUs usadas na simulação. O valor também é lido com a função ler_inteiro_com_padrao. O programa garante que o número de CPUs seja no mínimo 2 e no máximo MAX_CPUS, evitando configurações inválidas.

**Elemento
visual**

Função

Origem

Selecione o
algoritmo

Permite escolher o algoritmo de
escalonamento

printf na função main

ENTER para manter	Mantém o valor carregado do arquivo caso o usuário não digite nada	Leitura com fgets e verificação de <code>buffer[0] != '\n'</code>
1 - SRTF	Opção para selecionar o algoritmo Shortest Remaining Time First	case 1: <code>strcpy(config.algorithm, "SRTF")</code>
2 - PRIOP	Opção para selecionar Prioridade Preemptivo	case 2: <code>strcpy(config.algorithm, "PRIOP")</code>
Opcao:	Entrada digitada pelo usuário	Variável buffer, convertida com atoi
Opcao invalida	Aviso exibido se o usuário digitar uma opção diferente de 1 ou 2	Bloco default do switch
Quantum	Define o tempo máximo de execução antes de troca de contexto	Campo <code>config.quantum</code>
Numero de CPUs	Define quantos processadores serão usados na simulação	Campo <code>config.cpu_count</code>
Valor mínimo de quantum	Impede quantum menor que 1	<code>if (config.quantum < 1) config.quantum = 1</code>
Mínimo de 2 CPUs	Garante o requisito de múltiplos processadores	<code>if (config.cpu_count < 2)</code>
Máximo de CPUs	Impede ultrapassar o limite definido no programa	<code>MAX_CPUS</code>

Tela 4 — Configurações Decididas

```
--- Configuracoes finais ---  
Algoritmo : SRTF  
Quantum   : 5  
CPUs      : 2
```

Essa tela apresenta as configurações finais que serão utilizadas durante a execução do simulador. Os valores exibidos representam o resultado final após a leitura do arquivo `entrada.txt` e possíveis alterações realizadas pelo usuário na etapa de configuração inicial.

Elemento visual	Função	Origem
--- Configuracoes finais ---	Identifica a seção de confirmação das configurações finais	printf na função main
Algoritmo : %s	Exibe o algoritmo de escalonamento selecionado	Campo <code>config.algorithm</code>
Quantum : %d	Exibe o quantum utilizado na simulação	Campo <code>config.quantum</code>
CPUs : %d	Exibe a quantidade de CPUs configuradas	Campo <code>config.cpu_count</code>
Valores exibidos na tela	Confirmam as configurações efetivamente utilizadas na execução	Estrutura Config

Exibição das
informações

Realizada antes do início da simulação

Função main.c

Tela 5 — Lista de Tarefas Carregadas

```
Tarefas carregadas (3):  
T1   | Cor: #FF0000 | Ingresso: 0 | Duracao: 10 | Prioridade: 3 | Eventos: -  
T2   | Cor: #00FF00 | Ingresso: 0 | Duracao: 10 | Prioridade: 2 | Eventos: -  
T3   | Cor: #0000FF | Ingresso: 2 | Duracao: 4 | Prioridade: 5 | Eventos: -
```

Essa tela apresenta todas as tarefas carregadas pelo simulador a partir do arquivo entrada.txt. Cada linha representa uma tarefa que será utilizada durante a execução da simulação.

As informações exibidas são armazenadas no vetor de estruturas Task, preenchido pela função load_config localizada no arquivo config.c.

Elemento visual	Função	Origem da informação
Tarefas carregadas (%d)	Exibe a quantidade total de tarefas lidas do arquivo	Variável task_count
T%-3d	Identificador único da tarefa	Campo tasks[i].id
Cor: #%%s	Cor utilizada para representar visualmente a tarefa no gráfico de Gantt SVG	Campo tasks[i].color
Ingresso: %2d	Tick em que a tarefa entra no sistema	Campo tasks[i].arrival_time

Duracao: %2d	Tempo total necessário para execução da tarefa	Campo tasks[i].duration
Prioridade: %2d	Prioridade utilizada pelo algoritmo PRIOP	Campo tasks[i].priority
Eventos: %s	Lista de eventos associados à tarefa	Campo tasks[i].events
Vetor de tarefas	Estrutura que armazena todas as tarefas carregadas	Vetor Task tasks[]
Exibição das tarefas	Realizada antes do início da simulação	Função main.c

Tela 6 — Modo de Execução

```

Selecione o modo de execucao:
  1 - Completo    (executa tudo de uma vez)
  2 - Passo-a-passo (interativo, com Gantt ao vivo)
Opcao: 2

```

Essa tela permite ao usuário escolher o modo de execução da simulação. Antes da exibição do menu, o sistema inicializa todas as estruturas internas necessárias através da função `simulation_init`, responsável por preparar o estado inicial da simulação, incluindo CPUs, tarefas, histórico do Gantt e relógio global.

O simulador possui dois modos de execução:

- Modo Completo: executa toda a simulação automaticamente até o término das tarefas;
- Modo Passo-a-Passo: permite interação em tempo real, possibilitando avançar, retroceder, modificar tarefas e inspecionar o estado interno do sistema.

O programa também permite que o modo seja informado diretamente pela linha de comando através dos argumentos `completo` ou `passo`. Caso nenhum argumento seja informado, o menu é exibido no terminal para escolha manual do usuário.

Se o usuário informar uma opção inválida, o sistema utiliza automaticamente o modo completo como padrão.

Elemento visual	Função	Origem
Selecione o modo de execucao	Exibe o menu de escolha do modo de simulação	printf na função main
1 - Completo	Executa toda a simulação automaticamente	Variável mode = 1
2 - Passo-a-passo	Executa a simulação em modo interativo	Variável mode = 2
Opcao:	Entrada digitada pelo usuário	Leitura com fgets e conversão com atoi
Opcao invalida	Aviso exibido quando a opção escolhida é inválida	Verificação if (mode != 1 && mode != 2)
simulation_init	Inicializa o estado interno da simulação	Função simulation_init
Estrutura da simulação	Armazena CPUs, tarefas, clock e histórico	Struct SimulationState
Argumentos completo e passo	Permite definir o modo diretamente pela linha de comando	Verificação de argc e argv
Modo padrão	Define o modo completo caso a entrada seja inválida	mode = 1

Tela 7.1 — Modo de Execução Passoa-a-passo

```
=== Modo Passo-a-Passo ===  
Comandos: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
```

Essa tela representa o modo de execução interativo do simulador. Nesse modo, o usuário possui liberdade para controlar a execução da simulação em tempo real, podendo avançar, retroceder, modificar tarefas e inspecionar o estado interno do sistema operacional.

Os comandos disponíveis são exibidos diretamente no terminal e são processados pela função `simulation_run_step_by_step`, responsável pelo controle principal da execução interativa.

Comando	Função	Origem
n	Avança a simulação em um tick (next)	Função <code>simulation_run_step_by_step</code>
b	Retrocede a simulação para um snapshot anterior (back)	Função <code>simulation_run_step_by_step</code> e histórico <code>GanttHistory</code>
m	Modifica manualmente o estado e o tempo restante de uma tarefa (modify)	Função <code>prompt_modify_task</code>
i	Exibe o estado interno detalhado das tarefas e CPUs (inspect)	Função <code>inspect_state</code>

Tela 7.1.1 — Comando Next

```
Tick atual: 0 | Historico: 0 entradas
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:n

--- Tick 0 ---
[NOVA TAREFA] T0 chegou ao sistema no tick 0.
[NOVA TAREFA] T1 chegou ao sistema no tick 0.
[NOVA TAREFA] T2 chegou ao sistema no tick 0.
[SORTEIO] Desempate aleatorio ocorreu neste tick.
CPU 0 -> T0
CPU 1 -> T1

=== Grafico de Gantt | Tick: 0 ===

      | 0 |
-----+-----+
T2    |  | ↓t=0
T1    | C1* | ↓t=0
T0    | C0* | ↓t=0
-----+-----+
CPU0  | T0 |
CPU1  | T1 |

Legenda: Cx Exec.(CPU x)  Pronto  Suspenso  Nao chegou  --- CPU desligada
          ↓t=N Chegada no tick N  ✓t=N Fim no tick N  * Sorteio

Tick atual: 1 | Historico: 1 entradas
```

Essa tela representa o uso do comando `n`, responsável por avançar a simulação em um tick no modo passo-a-passo. Esse modo permite que o usuário acompanhe a execução do sistema gradualmente, observando a cada etapa o comportamento das tarefas, das CPUs e do escalonador.

O comando `n` é processado dentro da função `simulation_run_step_by_step`, localizada em `simulation.c`. Quando o usuário digita `n`, o simulador executa a função `simulation_step`, que realiza as operações de um tick da simulação, como verificar novas tarefas, aplicar o escalonamento, executar as tarefas selecionadas, atualizar os estados internos e registrar o histórico usado pelo gráfico de Gantt.

Elemento visual

Função

Origem

n	Avança a simulação em um tick	Comando lido em cmd[0] == 'n'
Tick atual	Indica o instante atual da simulação	Campo sim->clock
Historico: X entradas	Mostra a quantidade de snapshots armazenados	Campo sim->history.count
snapshot_pos	Controla se o usuário está navegando em um ponto antigo do histórico	Variável local snapshot_pos
sim->history.count = snapshot_pos + 1	Remove snapshots futuros após um retrocesso	Bloco executado quando snapshot_pos >= 0
restore_snapshot	Restaura o estado salvo antes de retomar a execução	Função restore_snapshot
sim->clock	Relógio global da simulação	Campo clock da struct SimulationState
simulation_step(sim)	Executa um tick completo da simulação	Função simulation_step
[SIMULACAO CONCLUIDA]	Indica que não há mais tarefas pendentes	Retorno falso de simulation_step
Resumo das tarefas	Exibe início, fim e turnaround das tarefas	Vetor sim->tasks[]

gantt_print_terminal	Atualiza o gráfico de Gantt no terminal	Função gantt_print_terminal
running = 0	Encerra o laço do modo passo-a-passo	Variável de controle running

.Tela 7.1.2 — Comando Back

```
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:b
Retrocedeu para tick 0.

=== Grafico de Gantt | Tick: 0 ===

      |  0 |
-----+-----+
T2    |  █  |  ↓t=0
T1    | C1 * |  ↓t=0
T0    | C0 * |  ↓t=0
-----+-----+
CPU0  |  T0  |
CPU1  |  T1  |

Legenda:  Cx  Exec.(CPU x)  █  Pronto  █  Suspenso  Nao chegou  ---- CPU desligada
          ↓t=N Chegada no tick N  ✓t=N Fim no tick N  * Sorteio

Tick atual: 0 | Historico: 2 entradas
```

Essa tela representa o uso do comando b, responsável por retroceder a simulação para um tick anterior no modo passo-a-passo. Esse recurso permite que o usuário volte no histórico da execução para analisar novamente um estado passado do sistema.

O comando b é processado dentro da função `simulation_run_step_by_step`, localizada em `simulation.c`. Ao ser acionado, o simulador calcula qual snapshot anterior deve ser restaurado. Cada snapshot armazena informações importantes do sistema naquele tick, como estado das tarefas, tempo restante, CPUs ativas e tarefas associadas a cada processador.

Após identificar o snapshot correto, a função `restore_snapshot` é chamada para restaurar o estado da simulação. Isso altera o relógio global, os estados das tarefas e a situação das CPUs, permitindo que o sistema volte exatamente ao ponto selecionado.

Depois do retrocesso, o gráfico de Gantt é exibido apenas até o tick restaurado, evitando que apareçam ticks futuros que já não fazem parte do estado atual da navegação.

Elemento visual	Função	Origem
b	Comando usado para retroceder a simulação	Leitura de <code>cmd[0] == 'b'</code>
Retrocedeu para tick X	Informa para qual tick o sistema voltou	<code>sim->history.entries[snapshot_pos].tick</code>
snapshot_pos	Guarda a posição atual dentro do histórico	Variável local <code>snapshot_pos</code>
target	Define o índice do snapshot anterior que será restaurado	Cálculo dentro do bloco do comando b
restore_snapshot	Restaura o estado do sistema salvo naquele tick	Função <code>restore_snapshot</code>
sim->clock	Relógio global ajustado para o tick restaurado	Campo <code>clock</code> da struct <code>SimulationState</code>
sim->history.count	Quantidade total de snapshots salvos	Estrutura <code>GanttHistory</code>
old_count	Guarda temporariamente o tamanho original do histórico	Variável local usada antes de limitar a exibição

sim->history.count = snapshot_pos + 1	Faz o Gantt mostrar somente até o tick restaurado	Ajuste temporário antes de chamar gantt_print_terminal
gantt_print_terminal	Exibe o gráfico de Gantt atualizado até o ponto restaurado	Função gantt_print_terminal
Nao ha historico para retroceder	Mensagem exibida quando não existe snapshot anterior	Condição if (target < 0)

Tela 7.1.3 — Comando Modify

```
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:m
Digite o ID da tarefa a modificar (ou -1 para cancelar): 1
Estado atual de T1: RUNNING | Tempo restante: 3
Novo estado (0=NEW, 1=READY, 2=RUNNING, 3=SUSPENDED, 4=FINISHED, -1=manter): 4
Estado de T1 alterado para FINISHED.

Tick atual: 0 | Historico: 2 entradas
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:m
Digite o ID da tarefa a modificar (ou -1 para cancelar): 2
Estado atual de T2: READY | Tempo restante: 4
Novo estado (0=NEW, 1=READY, 2=RUNNING, 3=SUSPENDED, 4=FINISHED, -1=manter): 2
Estado de T2 alterado para RUNNING.
T2 foi ajustada para READY porque nao estava em nenhuma CPU.
Novo tempo restante (atual=4, -1=manter): 3
Tempo restante de T2 alterado para 3.

Tick atual: 0 | Historico: 2 entradas
```

Essa tela representa o uso do comando m, responsável por permitir a modificação manual do estado de uma tarefa durante a execução da simulação. Esse recurso funciona como uma ferramenta de teste e depuração, permitindo ao usuário alterar dinamicamente o comportamento do sistema operacional multitarefa.

O comando `m` é processado na função `simulation_run_step_by_step`, que chama a função `prompt_modify_task`, localizada em `simulation.c`. Essa função solicita ao usuário o identificador da tarefa que será modificada e permite alterar:

- o estado da tarefa;
- o tempo restante de execução.

O sistema também realiza diversas validações para manter a consistência interna da simulação. Caso uma tarefa seja marcada como `FINISHED`, o simulador automaticamente:

- zera o tempo restante;
- remove a tarefa da CPU;
- encerra o slice de execução;
- registra o tick de finalização.

Além disso, o sistema impede estados inválidos, como uma tarefa marcada como `RUNNING` sem estar associada a uma CPU ativa.

Elemento visual	Função	Origem
<code>m</code>	Comando usado para modificar uma tarefa manualmente	Leitura de <code>cmd[0] == 'm'</code>
Digite o ID da tarefa	Solicita qual tarefa será alterada	Função <code>prompt_modify_task</code>
Estado atual de T%d	Exibe o estado atual da tarefa selecionada	Campo <code>sim->tasks[idx].state</code>
Tempo restante	Mostra quanto tempo ainda falta para finalizar a tarefa	Campo <code>remaining_time</code>
Novo estado	Permite escolher um novo estado para a tarefa	Entrada do usuário via <code>scanf</code>

0=NEW	Estado de tarefa ainda não ingressada	Enum NEW
1=READY	Estado de tarefa pronta para executar	Enum READY
2=RUNNING	Estado de tarefa em execução	Enum RUNNING
3=SUSPENDED	Estado de tarefa suspensa/bloqueada	Enum SUSPENDED
4=FINISHED	Estado de tarefa finalizada	Enum FINISHED
Estado de T%d alterado	Confirma a alteração realizada	printf em prompt_modify_task
Novo tempo restante	Permite alterar manualmente o remaining time	Campo remaining_time
remaining_time = 0	Executado automaticamente ao finalizar uma tarefa	Bloco if (new_state == FINISHED)
cpu_id = -1	Remove a tarefa da CPU ao finalizar ou suspender	Campo cpu_id
ticks_this_slice = 0	Reinicia o controle de quantum da tarefa	Campo ticks_this_slice
finish_time = sim->clock	Registra o tick em que a tarefa foi finalizada	Campo finish_time

RUNNING sem CPU	Validação para impedir estados inconsistentes	Verificação após alteração de estado
T%d foi ajustada para READY	Correção automática de estado inválido	printf de validação
Tarefa nao encontrada	Mensagem exibida caso o ID informado não exista	Verificação do índice idx
-1 para cancelar	Permite cancelar a modificação sem alterar a tarefa	Verificação if (id == -1)

Tela 7.1.4 — Comando Inspect

```
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:i

--- Estado do sistema no tick 0 ---
Tarefas:
  T0  | RUNNING  | Restante:  3 | Prioridade: 1 | CPU:  0 | Slice: 1/2
  T1  | FINISHED  | Restante:  0 | Prioridade: 1 | CPU: -1 | Slice: 0/2
  T2  | READY     | Restante:  3 | Prioridade: 1 | CPU: -1 | Slice: 0/2
CPUs:
  CPU 0 -> T0
  CPU 1 -> T1
---

Tick atual: 0 | Historico: 2 entradas
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
```

Essa tela representa o uso do comando `i`, responsável por exibir o estado interno detalhado do sistema durante a execução da simulação. Esse recurso funciona como um debugger do simulador, permitindo ao usuário acompanhar em tempo real as informações das tarefas e das CPUs.

O comando i é processado na função `simulation_run_step_by_step`, localizada em `simulation.c`, que chama a função `inspect_state`. Essa função percorre todas as tarefas e CPUs do sistema, exibindo informações importantes sobre o estado atual da simulação.

A tela de inspect permite visualizar:

- estado atual de cada tarefa;
- tempo restante;
- prioridade;
- CPU associada;
- quantum utilizado;
- estado de cada CPU;
- quantidade de ticks ociosos.

Essas informações são úteis para depuração, validação do escalonador e acompanhamento do comportamento do sistema operacional multitarefa ao longo da execução.

Elemento visual	Função	Origem
i	Comando utilizado para inspecionar o estado interno do sistema	Leitura de <code>cmd[0] == 'i'</code>
Estado do sistema no tick %d	Exibe o tick atual da simulação	Campo <code>sim->clock</code>
Tarefas:	Inicia a listagem das tarefas do sistema	Função <code>inspect_state</code>
T%-3d	Identificador único da tarefa	Campo <code>sim->tasks[i].id</code>
NEW	Tarefa ainda não ingressou no sistema	Enum NEW
READY	Tarefa pronta para executar	Enum READY

RUNNING	Tarefa atualmente em execução	Enum RUNNING
SUSPENDED	Tarefa suspensa/bloqueada	Enum SUSPENDED
FINISHED	Tarefa finalizada	Enum FINISHED
Restante: %3d	Tempo restante para conclusão da tarefa	Campo remaining_time
Prioridade: %2d	Prioridade utilizada pelo algoritmo PRIOP	Campo priority
CPU: %2d	CPU atualmente associada à tarefa	Campo cpu_id
Slice: %d/%d	Quantum já utilizado pela tarefa no processador	Campo ticks_this_slice e config.quantum
CPUs:	Inicia a listagem das CPUs do sistema	Função inspect_state
CPU %d -> T%d	Mostra qual tarefa está executando em cada CPU	Vetor sim->cpus[]
CPU desligada	Indica CPU sem tarefa atribuída	Campo active da CPU
%d ticks ociosa	Quantidade de ticks em que a CPU permaneceu sem execução	Campo idle_time
Vetor sim->tasks[]	Estrutura principal das tarefas da simulação	Struct Task

Vetor sim->cpus[]	Estrutura principal das CPUs simuladas	Struct CPU
estados[]	Vetor auxiliar usado para converter enum em texto	Array local da função inspect_state

Tela 7.1.5 — Comando Quit

```
Comandos disponíveis: [n]ext | [b]ack | [m]odify | [i]nspect | [q]uit
Digite o comando desejado:q

Tempo ocioso por CPU:
  CPU 0: 0 tick(s) desligada
  CPU 1: 0 tick(s) desligada
Grafico de Gantt SVG salvo em 'gantt.svg'.
Simulacao encerrada.
```

Essa tela representa o encerramento do modo passo-a-passo da simulação através do comando q (quit). Esse comando permite que o usuário finalize manualmente a execução interativa do simulador a qualquer momento.

O comando é processado dentro da função `simulation_run_step_by_step`, localizada no arquivo `simulation.c`. Quando o usuário digita q, a variável de controle `running` recebe o valor 0, encerrando o laço principal responsável pela interação do modo passo-a-passo.

Após sair do laço principal, o simulador executa automaticamente as rotinas finais de encerramento. Primeiramente, é exibido o tempo ocioso acumulado de cada CPU durante a simulação. Em seguida, o sistema gera o arquivo `gantt.svg`, contendo a representação gráfica completa do histórico de execução das tarefas. Por fim, o programa exibe a mensagem indicando o encerramento da simulação.

Essa tela representa o encerramento seguro da execução, garantindo que todas as informações finais sejam processadas e salvas corretamente antes do término do programa.

Elemento visual

Função

Origem

q	Comando utilizado para encerrar o modo passo-a-passo	Leitura de cmd[0] == 'q'
running = 0	Finaliza o laço principal da simulação interativa	Variável running
Tempo ocioso por CPU	Exibe o tempo total em que cada CPU permaneceu sem tarefas	Vetor sim->cpus[]
CPU %d	Identifica cada processador da simulação	Índice c do vetor de CPUs
%d tick(s) desligada	Quantidade de ticks ociosos acumulados pela CPU	Campo idle_time
gantt_save_svg	Gera o arquivo SVG do gráfico de Gantt	Função gantt_save_svg
gantt.svg	Arquivo gerado contendo a visualização gráfica da simulação	Nome definido na chamada da função
Simulacao encerrada	Mensagem final exibida ao usuário	printf ao final da função
Histórico da simulação	Base utilizada para gerar o gráfico SVG final	Estrutura sim->history
simulation_run_step_by_step	Função principal do modo interativo	Arquivo simulation.c

Tela 7.2 — Modo de Execução Completo

```
=== Modo de Execucao Completa ===

--- Tick 0 ---
[NOVA TAREFA] T0 chegou ao sistema no tick 0.
[NOVA TAREFA] T1 chegou ao sistema no tick 0.
[NOVA TAREFA] T2 chegou ao sistema no tick 0.
[SORTEIO]     Desempate aleatorio ocorreu neste tick.
CPU 0  -> T2
CPU 1  -> T1

--- Tick 1 ---
CPU 0  -> T2
CPU 1  -> T1

--- Tick 2 ---
CPU 0  -> T2
CPU 1  -> T1

--- Tick 3 ---
[CONCLUIDA]   T2 terminou no tick 3.
[CONCLUIDA]   T1 terminou no tick 3.
CPU 0  -> desligada (ocioso: 0 ticks)
CPU 1  -> desligada (ocioso: 0 ticks)
```

```
--- Tick 4 ---  
CPU 0  -> T0  
CPU 1  -> desligada (ocioso: 1 ticks)
```

```
--- Tick 5 ---  
CPU 0  -> T0  
CPU 1  -> desligada (ocioso: 2 ticks)
```

```
--- Tick 6 ---  
CPU 0  -> T0  
CPU 1  -> desligada (ocioso: 3 ticks)
```

```
--- Tick 7 ---  
[CONCLUIDA] T0 terminou no tick 7.  
CPU 0  -> desligada (ocioso: 0 ticks)  
CPU 1  -> desligada (ocioso: 4 ticks)
```

```
=== Simulacao concluida no tick 8 ===
```

```
Tempo ocioso por CPU:  
CPU 0: 0 tick(s) ociosa  
CPU 1: 4 tick(s) ociosa
```

```
Resumo das tarefas:  
T0 | inicio: 4 | fim: 7 | turnaround: 7  
T1 | inicio: 0 | fim: 3 | turnaround: 3  
T2 | inicio: 0 | fim: 3 | turnaround: 3
```



Essa tela representa o modo de execução automática do simulador. Nesse modo, a simulação é executada do início ao fim sem intervenção do usuário, avançando tick por tick até que não existam mais tarefas pendentes para executar.

A função responsável por essa tela é `simulation_run_complete`, localizada no arquivo `simulation.c`. Ela utiliza a função `simulation_step` dentro de um laço `while`, permitindo que o sistema continue executando enquanto houver tarefas disponíveis ou pendentes.

Elemento visual	Função	Origem
=== Modo de Execucao Completa ===	Indica que a simulação está sendo executada automaticamente	Função <code>simulation_run_complete</code>
Simulacao concluida no tick %d	Mostra o tick final da simulação	Campo <code>sim->clock</code>
Tempo ocioso por CPU	Exibe o tempo em que cada CPU ficou sem tarefa	Vetor <code>sim->cpus[c].idle_time</code>
CPU %d	Identifica cada processador do sistema simulado	Índice <code>c</code> no vetor <code>cpus</code>

tick(s) ociosa	Indica a quantidade de ticks em que a CPU ficou ociosa/desligada	Campo idle_time da CPU
Resumo das tarefas	Apresenta os principais dados finais de cada tarefa	Vetor sim->tasks[]
T%d	Identificador da tarefa	Campo sim->tasks[i].id
inicio	Tick em que a tarefa começou sua execução	Campo start_time da tarefa
fim	Tick em que a tarefa terminou sua execução	Campo finish_time da tarefa
turnaround	Tempo total entre chegada e finalização da tarefa	finish_time - arrival_time
Gráfico de Gantt no terminal	Mostra a execução completa das tarefas ao longo do tempo	Função gantt_print_terminal
Arquivo gantt.svg	Gera a visualização gráfica final em SVG	Função gantt_save_svg
Histórico da simulação	Base usada para montar o Gantt final	Estrutura sim->history